

```

//-----
//
// Name:      vadd.c
//
// Purpose:   Elementwise addition of two vectors (c = a + b)
//
// HISTORY:   Written by Tim Mattson, December 2009
//            Updated by Tom Deakin and Simon McIntosh-Smith, October 2012
//            Updated by Tom Deakin, July 2013
//            Updated by Tom Deakin, October 2014
//-----

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#ifdef __APPLE__
#include <OpenCL/opencl.h>
#include <unistd.h>
#else
#include <CL/cl.h>
#endif

#include "err_code.h"

//pick up device type from compiler command line or from
//the default type
#ifndef DEVICE
#define DEVICE CL_DEVICE_TYPE_DEFAULT
#endif

extern double wtime();          // returns time since some fixed past point (wtime.c)
extern int output_device_info(cl_device_id );

//-----

#define TOL      (0.001)      // tolerance used in floating point comparisons
#define LENGTH  (1024)      // length of vectors a, b, and c

//-----
//
// kernel:  vadd
//
// Purpose: Compute the elementwise sum c = a+b
//
// input:  a and b float vectors of length count
//
// output: c float vector of length count holding the sum a + b
//

const char *KernelSource = "\n" \
    "__kernel void vadd(                                \n" \
    "    __global float* a,                                \n" \
    "    __global float* b,                                \n" \
    "    __global float* c,                                \n" \
    "    const unsigned int count)                        \n" \
    "{                                                    \n" \
    "    int i = get_global_id(0);                        \n" \
    "    if(i < count)                                    \n" \
    "        c[i] = a[i] + b[i];                          \n" \
    "                                                    \n" \
    "};                                                    \n" \
    "\n";

//-----

int main(int argc, char** argv)
{
    int          err;          // error code returned from OpenCL calls

```

```

float*      h_a = (float*) calloc(LENGTH, sizeof(float));    // a vector
float*      h_b = (float*) calloc(LENGTH, sizeof(float));    // b vector
float*      h_c = (float*) calloc(LENGTH, sizeof(float));    // c vector (a+
b) returned from the compute device

unsigned int correct;    // number of correct results

size_t global;    // global domain size

cl_device_id device_id;    // compute device id
cl_context context;    // compute context
cl_command_queue commands;    // compute command queue
cl_program program;    // compute program
cl_kernel ko_vadd;    // compute kernel

cl_mem d_a;    // device memory used for the input a vector
cl_mem d_b;    // device memory used for the input b vector
cl_mem d_c;    // device memory used for the output c vector

// Fill vectors a and b with random float values
int i = 0;
int count = LENGTH;
for(i = 0; i < count; i++){
    h_a[i] = rand() / (float)RAND_MAX;
    h_b[i] = rand() / (float)RAND_MAX;
}

// Set up platform and GPU device

cl_uint numPlatforms;

// Find number of platforms
err = clGetPlatformIDs(0, NULL, &numPlatforms);
checkError(err, "Finding platforms");
if (numPlatforms == 0)
{
    printf("Found 0 platforms!\n");
    return EXIT_FAILURE;
}

// Get all platforms
cl_platform_id Platform[numPlatforms];
err = clGetPlatformIDs(numPlatforms, Platform, NULL);
checkError(err, "Getting platforms");

// Secure a GPU
for (i = 0; i < numPlatforms; i++)
{
    err = clGetDeviceIDs(Platform[i], DEVICE, 1, &device_id, NULL);
    if (err == CL_SUCCESS)
    {
        break;
    }
}

if (device_id == NULL)
    checkError(err, "Finding a device");

err = output_device_info(device_id);
checkError(err, "Printing device output");

// Create a compute context
context = clCreateContext(0, 1, &device_id, NULL, NULL, &err);
checkError(err, "Creating context");

// Create a command queue
commands = clCreateCommandQueue(context, device_id, 0, &err);
checkError(err, "Creating command queue");

// Create the compute program from the source buffer
program = clCreateProgramWithSource(context, 1, (const char **) & KernelSource,
NULL, &err);
checkError(err, "Creating program");

```

```

// Build the program
err = clBuildProgram(program, 0, NULL, NULL, NULL, NULL);
if (err != CL_SUCCESS)
{
    size_t len;
    char buffer[2048];

    printf("Error: Failed to build program executable!\n%s\n", err_code(err));
    clGetProgramBuildInfo(program, device_id, CL_PROGRAM_BUILD_LOG, sizeof(buffer),
r), buffer, &len);
    printf("%s\n", buffer);
    return EXIT_FAILURE;
}

// Create the compute kernel from the program
ko_vadd = clCreateKernel(program, "vadd", &err);
checkError(err, "Creating kernel");

// Create the input (a, b) and output (c) arrays in device memory
d_a = clCreateBuffer(context, CL_MEM_READ_ONLY, sizeof(float) * count, NULL,
&err);
checkError(err, "Creating buffer d_a");

d_b = clCreateBuffer(context, CL_MEM_READ_ONLY, sizeof(float) * count, NULL,
&err);
checkError(err, "Creating buffer d_b");

d_c = clCreateBuffer(context, CL_MEM_WRITE_ONLY, sizeof(float) * count, NULL,
&err);
checkError(err, "Creating buffer d_c");

// Write a and b vectors into compute device memory
err = clEnqueueWriteBuffer(commands, d_a, CL_TRUE, 0, sizeof(float) * count, h_a
, 0, NULL, NULL);
checkError(err, "Copying h_a to device at d_a");

err = clEnqueueWriteBuffer(commands, d_b, CL_TRUE, 0, sizeof(float) * count, h_b
, 0, NULL, NULL);
checkError(err, "Copying h_b to device at d_b");

// Set the arguments to our compute kernel
err = clSetKernelArg(ko_vadd, 0, sizeof(cl_mem), &d_a);
err = clSetKernelArg(ko_vadd, 1, sizeof(cl_mem), &d_b);
err = clSetKernelArg(ko_vadd, 2, sizeof(cl_mem), &d_c);
err = clSetKernelArg(ko_vadd, 3, sizeof(unsigned int), &count);
checkError(err, "Setting kernel arguments");

double rtime = wtime();

// Execute the kernel over the entire range of our 1d input data set
// letting the OpenCL runtime choose the work-group size
global = count;
err = clEnqueueNDRangeKernel(commands, ko_vadd, 1, NULL, &global, NULL, 0, NULL,
NULL);
checkError(err, "Enqueueing kernel");

// Wait for the commands to complete before stopping the timer
err = clFinish(commands);
checkError(err, "Waiting for kernel to finish");

rtime = wtime() - rtime;
printf("\nThe kernel ran in %lf seconds\n", rtime);

// Read back the results from the compute device
err = clEnqueueReadBuffer(commands, d_c, CL_TRUE, 0, sizeof(float) * count, h_c
, 0, NULL, NULL);
if (err != CL_SUCCESS)
{
    printf("Error: Failed to read output array!\n%s\n", err_code(err));
    exit(1);
}

```

```

// Test the results
correct = 0;
float tmp;

for(i = 0; i < count; i++)
{
    tmp = h_a[i] + h_b[i];    // assign element i of a+b to tmp
    tmp -= h_c[i];           // compute deviation of expected and output result
    if(tmp*tmp < TOL*TOL)    // correct if square deviation is less than tolerance squared
        correct++;
    else {
        printf(" tmp %f h_a %f h_b %f h_c %f \n",tmp, h_a[i], h_b[i], h_c[i]);
    }
}

// summarise results
printf("C = A+B:  %d out of %d results were correct.\n", correct, count);

// cleanup then shutdown
clReleaseMemObject(d_a);
clReleaseMemObject(d_b);
clReleaseMemObject(d_c);
clReleaseProgram(program);
clReleaseKernel(ko_vadd);
clReleaseCommandQueue(commands);
clReleaseContext(context);

free(h_a);
free(h_b);
free(h_c);

return 0;
}

```